SAVEETHA SCHOOL OF ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

COMPUTER SCIENCE AND ENGINEERING

CSA47 – DEEP LEARNING

ASSIGNMENT 2

CO2 – DEEP FEED-FORWARD AND CONVOLUTIONAL NETWORK DESIGN

Nature: Individual | Duration: 1 Week | Total Marks: 50

Name: Pola Nandivardhan **Register No.:** 192324270

CO2: Apply architectural design principles and optimization techniques to construct and train deep feed forward and convolutional neural networks.

Topics: Deep Feed-Forward Networks, Gradient-Based Learning, Back-Propagation, Regularization and Convolutional Networks.

Purpose: Design, train and analyze neural-network architectures rather than merely reproduce a standard model.

Scenario – Smart Agriculture and Crop Disease Detection: An agricultural research centre wants to predict crop yield using rainfall, temperature, soil moisture and fertilizer quantity. It also plans to classify healthy and diseased crop leaves using images. You are required to design, train, optimize and evaluate suitable deep neural network architectures.

TASK 1 – Deep Feed-Forward Network Architecture | 5 Marks

> An agricultural centre wants to predict crop yield using rainfall, temperature, soil moisture and fertilizer quantity. A feed-forward network has two hidden layers containing 8 and 4 neurons.
>
> a) Design the network by identifying the input, hidden and output layers and the number of neurons in each layer.
>
> b) Specify the dimensions of the weight matrices and bias vectors.
>
> c) Explain how inputs flow through the network to produce a crop-yield prediction.
>
> Evidence Expected: Network architecture, neuron counts, weight and bias dimensions, mathematical representation and forward propagation.

Answer

a) Network design

The problem is a regression task: four numerical inputs are mapped to one continuous output (crop yield). Using the two hidden layers given in the question, the architecture is **4 – 8 – 4 – 1** (a fully connected deep feed-forward network).

Layer	Neurons	Activation	Role
Input layer	4	–	x_1 = rainfall, x_2 = temperature, x_3 = soil moisture, x_4 = fertilizer quantity (all normalised)
Hidden layer 1	8	ReLU	Learns 8 non-linear combinations of the inputs
Hidden layer 2	4	ReLU	Combines these into 4 higher-level features
Output layer	1	Linear	Produces the predicted crop yield $\hat{y}$ (any real value)

A linear (identity) output activation is chosen because yield is an unbounded continuous value; ReLU in the hidden layers gives non-linearity and avoids vanishing gradients. Inputs are normalised (min–max or standardisation) so that features with large ranges, such as rainfall in mm, do not dominate.

b) Dimensions of weights and biases

For a layer with n_{in} inputs and n_{out} neurons, the weight matrix has shape ($n_{out} \times n_{in}$) and the bias vector has shape ($n_{out} \times 1$). Using column vectors:

Connection	Weight matrix	Bias vector	Parameters
Input → Hidden 1	$W_1 : 8 \times 4$	$b_1 : 8 \times 1$	32 + 8 = 40
Hidden 1 → Hidden 2	$W_2 : 4 \times 8$	$b_2 : 4 \times 1$	32 + 4 = 36
Hidden 2 → Output	$W_3 : 1 \times 4$	$b_3 : 1 \times 1$	4 + 1 = 5
Total			81 trainable parameters

c) Forward propagation

The input vector x (4 × 1) is passed layer by layer. In each layer a weighted sum plus bias is computed and then an activation function is applied:

$$z_1 = W_1 x + b_1 \qquad h_1 = \mathrm{ReLU}(z_1) \qquad (8 \times 1)$$

$$z_2 = W_2 h_1 + b_2 \qquad h_2 = \mathrm{ReLU}(z_2) \qquad (4 \times 1)$$

$$\hat{y} = W_3 h_2 + b_3 \qquad (1 \times 1)$$

Step 1: the four normalised measurements enter the input layer. Step 2: each of the 8 neurons of hidden layer 1 computes its weighted sum of the four inputs, adds its bias and applies ReLU. Step 3: each of the 4 neurons of hidden layer 2 does the same with the 8 outputs of layer 1. Step 4: the single output neuron forms a weighted sum of the 4 outputs of layer 2 and returns the predicted yield ŷ. The whole expression is the composition $\hat{y} = f_3(f_2(f_1(x)))$. For a mini-batch of m samples the same equations are applied with X of shape (m × 4) so that all samples are processed at once.

Code:

```
import numpy as np
np.random.seed(42)

def relu(z):
    return np.maximum(0, z)

# One normalised sample: [rainfall, temperature, soil moisture, fertilizer]
x = np.array([[0.6], [0.8], [0.5], [0.7]])          # shape (4, 1)

# Parameters of the 4-8-4-1 network
W1 = np.random.randn(8, 4) * 0.1;  b1 = np.zeros((8, 1))
W2 = np.random.randn(4, 8) * 0.1;  b2 = np.zeros((4, 1))
W3 = np.random.randn(1, 4) * 0.1;  b3 = np.zeros((1, 1))

# Forward propagation
z1 = W1 @ x + b1;   h1 = relu(z1)
z2 = W2 @ h1 + b2;  h2 = relu(z2)
y_hat = W3 @ h2 + b3                                # linear output (regression)

print("W1:", W1.shape, " b1:", b1.shape)
print("W2:", W2.shape, " b2:", b2.shape)
print("W3:", W3.shape, " b3:", b3.shape)
print("Total parameters:", sum(p.size for p in (W1, b1, W2, b2, W3, b3)))
print("Predicted yield (untrained network):", y_hat.item())
```

Output:

```
>>
================== RESTART: C:/All Folders/Deep Learning/lab1.py ==============
W1: (8, 4)  b1: (8, 1)
W2: (4, 8)  b2: (4, 1)
W3: (1, 4)  b3: (1, 1)
Total parameters: 81
Predicted yield (untrained network): 0.0023883509745172404
>>
```

Code:

```
from tensorflow.keras import Sequential, layers

model = Sequential([
    layers.Input(shape=(4,)),
    layers.Dense(8, activation="relu"),     # hidden layer 1
    layers.Dense(4, activation="relu"),     # hidden layer 2
    layers.Dense(1)                         # output layer (linear)
])
model.summary()
```

Output of model.summary() :

```
================== RESTART: C:/All Folders/Deep Learning/lab1.py ================
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 8)	40
dense_1 (Dense)	(None, 4)	36
dense_2 (Dense)	(None, 1)	5

```
Total params: 81 (324.00 B)
Trainable params: 81 (324.00 B)
Non-trainable params: 0 (0.00 B)
```

TASK 2 – Forward Propagation and Activation Functions | 2 Marks

> A neuron receives normalized inputs 0.6, 0.8 and 0.5. Its weights are 0.4, 0.3 and 0.2, and its bias is 0.1.
>
> a) Calculate the weighted sum received by the neuron.
>
> b) Apply ReLU and interpret the output.

> Evidence Expected: Weighted-sum calculation, activation-function application and interpretation.

Answers

a) Weighted sum

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

$$z = (0.4 \times 0.6) + (0.3 \times 0.8) + (0.2 \times 0.5) + 0.1$$

$$z = 0.24 + 0.24 + 0.10 + 0.10 = \mathbf{0.68}$$

b) ReLU and interpretation

$$a = \text{ReLU}(z) = \max(0, z) = \max(0, 0.68) = \mathbf{0.68}$$

Since z = 0.68 is positive, ReLU passes it through unchanged. The neuron is therefore **active** and sends the value 0.68 to the next layer. Had z been negative, ReLU would output 0 and the neuron would be switched off for this input. The calculation also shows that the first input contributes most to the sum because it is multiplied by the largest weight (0.4), and the bias shifts the neuron's activation threshold upward by 0.1.

Code:

```
import numpy as np

x = np.array([0.6, 0.8, 0.5])      # normalised inputs
w = np.array([0.4, 0.3, 0.2])      # weights
b = 0.1                            # bias

z = np.dot(w, x) + b               # weighted sum
a = max(0, z)                      # ReLU activation

print("Weighted sum z =", round(z, 4))
print("ReLU output a  =", round(a, 4))
```

Output:

```
================== RESTART: C:/All Folders/Deep Learning/lab1.py ==
Weighted sum z = 0.68
ReLU output a  = 0.68
>
```

TASK 3 – Gradient-Based Learning and Loss Calculation | 6 Marks

> A crop-yield network predicts 42 units when the actual yield is 50 units.
>
> a) Explain why a loss function is required and select a suitable loss function for regression.

b) Calculate the squared error loss using $L = (y - \hat{y})^2$.

c) Explain what the loss value indicates about the prediction.

d) Describe how gradient-based learning uses the loss to update network parameters.

Evidence Expected: Suitable loss selection, numerical calculation, interpretation and explanation of gradient-based optimization.

Answers

a) Need for a loss function and choice for regression

A neural network learns by correcting its mistakes, and a loss function converts "how wrong the prediction is" into a single number. It is required because: (i) it gives a measurable objective that training tries to minimise; (ii) it is differentiable, so gradients with respect to every weight can be computed; (iii) it allows different models and training epochs to be compared; and (iv) it tells the optimiser which direction improves the model.

For a regression problem such as crop yield, the **Mean Squared Error (MSE)** is suitable:

$$MSE = (1/N) \Sigma (y_i - \hat{y}_i)^2$$

MSE is smooth and differentiable everywhere, has a simple gradient, and penalises large errors more strongly than small ones. (MAE or Huber loss are alternatives when outliers are common.) For a single sample it reduces to the squared error $L = (y - \hat{y})^2$.

b) Squared error loss

$$L = (y - \hat{y})^2 = (50 - 42)^2 = 8^2 = \mathbf{64}$$

The gradient of the loss with respect to the prediction is $\partial L/\partial \hat{y} = -2(y - \hat{y}) = -2(8) = \mathbf{-16}$.

c) Interpretation

The prediction is 8 units below the true yield (an under-estimate of 16%). The loss of 64 is non-zero, so the network is not yet accurate; a perfect prediction would give $L = 0$. Squaring means that the loss grows quickly with the size of the error (an error of 16 would give 256, not 128), so the network is pushed hard to remove large mistakes. The loss does not show the direction of the error, but the negative gradient (−16) does: it shows that increasing $\hat{y}$ would reduce the loss. The square root of the loss (8 units) gives the error in the original yield unit.

d) Gradient-based learning

Gradient-based learning adjusts the parameters θ (all weights and biases) so that the loss becomes smaller:

- **Forward pass:** compute the prediction $\hat{y}$ and the loss L.
- **Gradient computation:** use back-propagation to find $\partial L/\partial \theta$ for every parameter; the gradient shows how much and in which direction L changes when a parameter changes.
- **Parameter update:** move each parameter a small step against its gradient, $\theta \leftarrow \theta - \eta \, \partial L/\partial \theta$, where η is the learning rate.
- **Repeat** over many mini-batches and epochs until the loss stops decreasing (convergence).

Geometrically, the loss is a surface over the parameter space and gradient descent walks downhill on it towards a minimum.

Code:

```
y, y_hat = 50, 42                # actual and predicted yield

error = y - y_hat
loss = error ** 2                # squared error loss
grad = -2 * (y - y_hat)          # dL/dy_hat

print("Error          :", error)
print("Squared error L  :", loss)
print("RMSE (same units):", abs(error))
print("dL/dy_hat       :", grad)
```

Output :

```
================= RESTART: C:/All Folders/Deep Learning/lab1.py ======
Error             : 8
Squared error L   : 64
RMSE (same units): 8
dL/dy_hat         : -16
>
```

TASK 4 – Gradient Descent and Weight Update | 4 Marks

> During training, a network weight is 0.5, its loss gradient is 0.8, and the learning rate is 0.1.
>
> a) Calculate the updated weight using the gradient descent rule.
>
> b) Explain why the update is in the opposite direction to the gradient.
>
> c) Discuss the effects of an extremely small or large learning rate.
>
> Evidence Expected: Gradient descent formula, numerical update, update-direction explanation and learning-rate analysis.

Answers

a) Updated weight

$$w_{new} = w_{old} - \eta \, (\partial L / \partial w)$$

$$w_{new} = 0.5 - (0.1 \times 0.8) = 0.5 - 0.08 = \mathbf{0.42}$$

b) Why the update opposes the gradient

The gradient $\partial L/\partial w$ points in the direction in which the loss **increases** most steeply. Because the aim is to **reduce** the loss, the weight must move in the opposite direction. Here the gradient is positive (0.8), meaning that increasing w would increase the loss, so w is

decreased from 0.5 to 0.42. If the gradient were negative, the minus sign would turn the update into an increase of w. For a small step, the first-order change in loss is $\Delta L \approx -\eta(\partial L/\partial w)^2$, which is always negative, so each update lowers the loss (provided η is small enough).

c) Effect of very small and very large learning rates

Learning rate	Effect on training	Result for this weight
Extremely small (η = 0.0001)	Tiny steps, very slow convergence, many epochs and high computation; may stall on plateaus or saddle points.	w = 0.5 − 0.00008 = 0.49992
Suitable (η = 0.1)	Steady, stable decrease of the loss.	w = 0.42
Extremely large (η = 5)	Steps overshoot the minimum; the loss oscillates, increases or diverges (NaN values).	w = 0.5 − 4.0 = −3.5 (large jump)

Hence the learning rate is one of the most important hyper-parameters: it must be small enough for stability but large enough for reasonable speed, and is often reduced during training with a schedule.

Code:

```
w, grad = 0.5, 0.8

for lr in [0.0001, 0.1, 5.0]:
    w_new = w - lr * grad
    print(f"learning rate = {lr:<7} ->  updated weight = {w_new:.5f}")
```

Output:

```
================== RESTART: C:/All Folders/Deep Learning/lab1.py ===========
learning rate = 0.0001  ->  updated weight = 0.49992
learning rate = 0.1     ->  updated weight = 0.42000
learning rate = 5.0     ->  updated weight = -3.50000
```

Ln

TASK 5 – Back-Propagation Through Multiple Layers | 3 Marks

> A crop-yield prediction network contains an input layer, two hidden layers and an output layer. Its prediction differs substantially from the actual yield.
>
> a) Explain how backpropagation determines the contribution of weights to the prediction error.
>
> b) Explain the role of the chain rule in calculating gradients across layers.

> c) Describe how gradients are used to update weights.
>
> Evidence Expected: Backpropagation process, chain-rule explanation, gradient flow and weight updates.

Answers

a) How back-propagation assigns responsibility for the error

Back-propagation first runs a forward pass (storing z and h of every layer) and computes the loss L. It then sends the error **backwards** from the output layer to the first layer. At each layer it computes an error signal $\delta = \partial L/\partial z$ that says how sensitive the loss is to that layer's weighted sums. For the network $x \rightarrow h_1 \rightarrow h_2 \rightarrow \hat{y}$ with squared error loss:

$$\delta_3 = \partial L/\partial \hat{y} = -2(y - \hat{y})$$

$$\delta_2 = (W_3^T\, \delta_3) \odot \mathrm{ReLU}'(z_2)$$

$$\delta_1 = (W_2^T\, \delta_2) \odot \mathrm{ReLU}'(z_1)$$

The gradient of a weight is its error signal multiplied by the activation that fed into it: $\partial L/\partial W_3 = \delta_3 h_2^T$, $\partial L/\partial W_2 = \delta_2 h_1^T$, $\partial L/\partial W_1 = \delta_1 x^T$, and $\partial L/\partial b_l = \delta_l$. A large gradient magnitude means the weight contributed strongly to the error, and a small one means it contributed little. The error is therefore shared out among all weights in proportion to their influence, and each layer reuses the quantities already computed for the layer above, making the process efficient.

b) Role of the chain rule

The loss depends on an early-layer weight only through a long chain of intermediate functions. The chain rule multiplies the local derivatives along that chain:

$$\partial L/\partial W_1 = (\partial L/\partial \hat{y})(\partial \hat{y}/\partial h_2)(\partial h_2/\partial z_2)(\partial z_2/\partial h_1)(\partial h_1/\partial z_1)(\partial z_1/\partial W_1)$$

Each factor is simple to compute (a weight matrix, an activation derivative or a loss derivative). The chain rule lets these factors be combined layer by layer, so gradients flow from the output to the input, and it explains why very small or very large local derivatives can cause vanishing or exploding gradients in deep networks.

c) Using the gradients to update weights

Once all gradients are available, every parameter is updated with the gradient-descent rule (or an optimiser such as Adam that uses the same gradients):

$$W_l \leftarrow W_l - \eta\, \partial L/\partial W_l \qquad b_l \leftarrow b_l - \eta\, \partial L/\partial b_l \qquad (l = 1, 2, 3)$$

The update lowers the loss for the current sample or mini-batch, so the next forward pass gives a prediction closer to the actual yield. Repeating forward pass, back-propagation and update over many iterations reduces the large prediction error gradually.

TASK 6 – Activation Functions and Vanishing/Exploding Gradients | 3 Marks

> A deep network for crop analysis learns slowly because gradients reaching early layers become extremely small.

> a) Explain how activation functions can contribute to vanishing gradients.
>
> b) Compare sigmoid, tanh and ReLU for hidden layers.
>
> c) Explain how activation-function choice affects deep-network training.
>
> Evidence Expected: Activation-function comparison, gradient behaviour and impact on training.

Answers

a) How activation functions cause vanishing gradients

During back-propagation the gradient reaching layer k is a product of many terms, including the activation derivatives of all later layers. The sigmoid has a maximum derivative of only 0.25, and tanh has a maximum of 1 that falls quickly away from zero. For large positive or negative inputs both functions **saturate** (the curve becomes flat) and the derivative is almost 0. Multiplying many numbers smaller than 1 shrinks the gradient exponentially with depth; for example, $0.25^{10} \approx 9.5 \times 10^{-7}$ after only ten sigmoid layers. The early layers then receive almost no learning signal, their weights barely change, and training becomes very slow. (The opposite problem, exploding gradients, occurs when the multiplied terms are larger than 1, e.g. large weights, giving huge unstable updates.)

b) Comparison of sigmoid, tanh and ReLU

Property	Sigmoid	Tanh	ReLU
Formula	$1 / (1 + e^{-z})$	$(e^{z} - e^{-z}) / (e^{z} + e^{-z})$	$\max(0, z)$
Output range	$(0, 1)$	$(-1, 1)$	$[0, \infty)$
Maximum derivative	0.25	1.0	1 (for $z > 0$)
Zero-centred	No	Yes	No
Saturation	Both sides	Both sides	Only negative side (gradient = 0)
Vanishing gradient	Severe	Moderate	Not for $z > 0$
Computation	Expensive (exponential)	Expensive (exponential)	Very cheap
Main drawback	Slow learning, non-zero-centred	Still saturates	“Dead neurons” if z stays negative
Typical use	Output of binary classifier	Some RNN / shallow networks	Default for hidden layers

c) Effect of activation choice on deep-network training

The activation function decides how well gradients survive the trip back through many layers. Sigmoid or tanh in deep hidden stacks produce small gradients, slow convergence and poor learning in early layers. ReLU has a gradient of exactly 1 for positive inputs, so the gradient

passes unchanged, training is faster and deeper networks become trainable; it is also cheap to compute and produces sparse activations. Its weakness is dead neurons, which Leaky ReLU, ELU or GELU reduce. Good practice is to combine ReLU-type activations with He initialisation, batch normalisation and residual connections, and to use gradient clipping against exploding gradients.

Code:

```
import numpy as np

def sigmoid(z):   return 1 / (1 + np.exp(-z))
def d_sigmoid(z): return sigmoid(z) * (1 - sigmoid(z))
def d_tanh(z):    return 1 - np.tanh(z) ** 2
def d_relu(z):    return (z > 0).astype(float)

print("Maximum derivative -> sigmoid: %.2f | tanh: %.2f | ReLU: %.2f"
      % (d_sigmoid(0.0), d_tanh(0.0), d_relu(np.array([1.0]))[0]))

np.random.seed(0)
depth = 10
z = np.random.randn(depth) * 1.5          # pre-activations of 10 layers

print("\nGradient factor accumulated over", depth, "layers:")
print("sigmoid :", np.prod(d_sigmoid(z)))
print("tanh    :", np.prod(d_tanh(z)))
print("ReLU    :", np.prod(d_relu(np.abs(z))), "(units on an active path, z > 0)")
```

Output :

```
>>>
================== RESTART: C:/All Folders/Deep Learning/lab1.py ================
Maximum derivative -> sigmoid: 0.25 | tanh: 1.00 | ReLU: 1.00

Gradient factor accumulated over 10 layers:
sigmoid : 1.2523235749821943e-09
tanh    : 4.916839389680443e-09
ReLU    : 1.0 (units on an active path, z > 0)
>>>
```

TASK 7 – Back-Propagation and Optimization Strategy | 4 Marks

> In one experiment, the training loss decreases slowly. In another, it decreases quickly but fluctuates considerably.
>
> a) Explain how the learning rate affects optimization.
>
> b) Explain how backpropagation and gradient descent work together in one iteration.
>
> c) Suggest changes to improve convergence in both experiments.
>
> d) Explain why the loss curve should be monitored.

> Evidence Expected: Relationship between backpropagation and optimization, learning-rate analysis, suitable adjustments and loss-curve interpretation.

Answers

a) Effect of the learning rate

The learning rate η sets the step size of every update. If η is too small, each step is tiny, the loss falls slowly and training may stall on flat regions. If η is too large, steps overshoot the minimum, so the loss falls quickly at first but then oscillates or even diverges. A well-chosen η gives fast and stable decrease. Therefore Experiment 1 (slow decrease) most likely uses a learning rate that is too small, and Experiment 2 (fast but fluctuating) most likely uses one that is too large (or has very noisy mini-batch gradients).

b) Back-propagation and gradient descent in one iteration

The two methods have different jobs: back-propagation **computes** the gradients and gradient descent **uses** them to change the weights. One iteration is:

- Take a mini-batch and perform the forward pass to get predictions $\hat{y}$.
- Compute the loss L between $\hat{y}$ and the actual yield.
- Back-propagation: apply the chain rule from the output layer to the first layer to obtain $\partial L/\partial W$ and $\partial L/\partial b$ for every layer.
- Gradient descent: update every parameter, $\theta \leftarrow \theta - \eta\, \partial L/\partial \theta$.
- Move to the next mini-batch (one pass over all mini-batches is one epoch).

c) Changes to improve convergence

Experiment	Diagnosis	Suggested changes
1: slow decrease	Learning rate too small; possible poor scaling, bad initialisation or vanishing gradients	• Increase η (use a learning-rate range test) • Use Adam or momentum • Normalise inputs; use He initialisation and ReLU • Add batch normalisation
2: fast but fluctuating	Learning rate too large and/or noisy gradients from small batches	• Decrease η or use learning-rate decay / cosine schedule • Increase the batch size • Use momentum, Adam or RMSProp • Apply gradient clipping

d) Why the loss curve must be monitored

The training and validation loss curves are the main diagnostic tool. They show whether training is converging, too slow (small η), unstable (large η) or diverging; whether the model is under-fitting (both losses high) or over-fitting (training loss falls while validation loss rises); when a plateau is reached so that the learning rate can be reduced; when to stop early

and which checkpoint to keep; and they can reveal bugs such as wrong labels or data leakage. Without monitoring, hyper-parameter changes would be guesswork.

Code:

```
import numpy as np
np.random.seed(1)

relu = lambda z: np.maximum(0, z)
x, y = np.array([[0.6], [0.8], [0.5], [0.7]]), 0.5       # one training sample
W1 = np.random.randn(8, 4) * 0.5;  b1 = np.zeros((8, 1))
W2 = np.random.randn(4, 8) * 0.5;  b2 = np.zeros((4, 1))
W3 = np.random.randn(1, 4) * 0.5;  b3 = np.zeros((1, 1))
lr = 0.05

def forward():
    z1 = W1 @ x + b1;   h1 = relu(z1)
    z2 = W2 @ h1 + b2;  h2 = relu(z2)
    return z1, h1, z2, h2, W3 @ h2 + b3

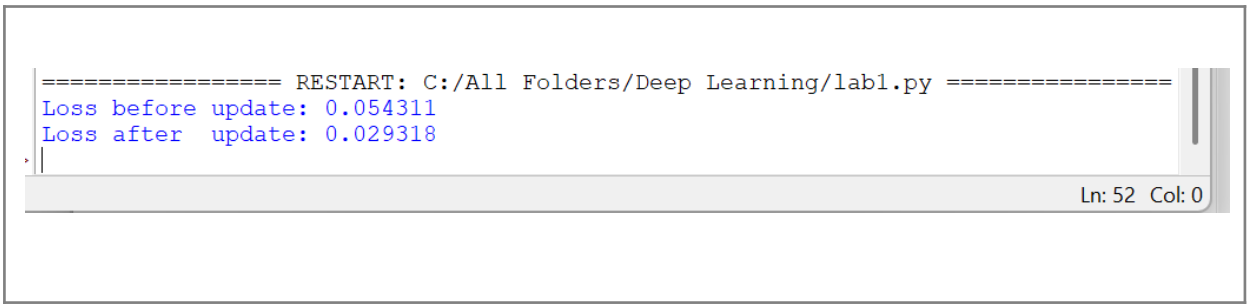
# 1) Forward pass and loss
z1, h1, z2, h2, y_hat = forward()
loss_before = ((y - y_hat) ** 2).item()

# 2) Backpropagation (chain rule, output -> input)
d3 = -2 * (y - y_hat)                    # dL/dz3
dW3, db3 = d3 @ h2.T, d3
d2 = (W3.T @ d3) * (z2 > 0)              # dL/dz2
dW2, db2 = d2 @ h1.T, d2
d1 = (W2.T @ d2) * (z1 > 0)              # dL/dz1
dW1, db1 = d1 @ x.T, d1

# 3) Gradient-descent update
W3 -= lr * dW3;  b3 -= lr * db3
W2 -= lr * dW2;  b2 -= lr * db2
W1 -= lr * dW1;  b1 -= lr * db1

# 4) Loss after one iteration
loss_after = ((y - forward()[-1]) ** 2).item()
print("Loss before update:", round(loss_before, 6))
print("Loss after  update:", round(loss_after, 6))
```

Output (paste your output here):



Code:

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(0)
rates = [0.001, 0.1, 0.9, 1.05]          # too small, good, too noisy, diverging
plt.figure(figsize=(7, 4))

for lr in rates:
    w, losses = 5.0, []
    for _ in range(40):
        losses.append(w ** 2)                    # loss L(w) = w^2
        grad = 2 * w + rng.normal(0, 1.0)        # noisy (mini-batch) gradient
        w = w - lr * grad
    plt.plot(losses, label=f"learning rate = {lr}")

plt.yscale("log")
plt.xlabel("Iteration");  plt.ylabel("Loss (log scale)")
plt.title("Effect of learning rate on the loss curve")
plt.legend();  plt.grid(alpha=0.3);  plt.show()
```

Output – loss-curve plot :



TASK 8 – Regularization and Overfitting Control | 4 Marks

> A crop-yield network achieves 99% training accuracy on a crop-category task but only 78% validation accuracy.
>
> a) Identify the likely learning problem and explain the evidence.
>
> b) Explain how L1 and L2 regularization control model complexity.
>
> c) Write a regularized loss function using an L2 penalty and explain the regularization parameter.
>
> d) Explain how dropout may improve generalization.
>
> Evidence Expected: Identification of overfitting, regularization explanation, loss formulation and dropout-based solution.

Answers

a) Learning problem

The network is **overfitting** (high variance). Training accuracy is 99% but validation accuracy is only 78%, a generalisation gap of 21 percentage points. The model has memorised the training examples, including their noise, rather than learning patterns that transfer to unseen data. Typical causes are too many parameters for the amount of data, long training without regularisation and noisy features. The evidence on the loss curves would be a training loss that keeps falling while the validation loss stops improving and starts to rise.

b) L1 and L2 regularization

Regularisation adds a penalty on the size of the weights to the loss, so the network can only use large weights when they clearly reduce the error. This restricts the effective complexity of the model.

- **L1 (Lasso):** penalty $\lambda\Sigma|w_i|$. It drives many weights exactly to zero, producing a sparse network and acting as automatic feature selection.
- **L2 (Ridge / weight decay):** penalty $\lambda\Sigma w_i^2$. It shrinks all weights smoothly towards zero without making them exactly zero, giving a smoother function that is less sensitive to noise in individual inputs.

c) L2-regularized loss function

$$L_{reg} = (1/N)\ \Sigma\ (y_i - \hat{y}_i)^2 + \lambda\ \Sigma_j\ w_j^2$$

(For classification the first term is replaced by cross-entropy loss.) The first term measures the data error and the second term penalises large weights. **λ (lambda)** is the regularisation parameter that controls the trade-off: $\lambda = 0$ means no regularisation (overfitting is likely); a very large λ forces the weights towards zero and causes under-fitting; a moderate value (for example 10^{-4} to 10^{-2}) is selected with the validation set. The gradient gains an extra term $2\lambda w$, so each update becomes $w \leftarrow w(1 - 2\eta\lambda) - \eta\ \partial L/\partial w$, which is why L2 is also called weight decay. Biases are normally not penalised.

d) Dropout

Dropout randomly sets a fraction p of the neurons in a layer to zero (e.g. p = 0.2–0.5) at every training iteration, and the remaining activations are scaled by 1/(1 − p). Because a neuron cannot depend on specific other neurons being present, co-adaptation is prevented and each neuron must learn useful, robust features. Training with dropout is like training a large ensemble of thinned sub-networks that share weights. At test time all neurons are used, which approximates averaging the ensemble and improves generalisation. Other useful remedies are early stopping, data augmentation, collecting more data and reducing the network size.

Code:

```
import numpy as np, tensorflow as tf
from tensorflow.keras import Sequential, layers, regularizers, callbacks

np.random.seed(0);  tf.random.set_seed(0)

# Synthetic noisy crop-category data (replace with the real dataset)
X = np.random.rand(1500, 4).astype("float32")
score = 2 * X[:, 0] + X[:, 2] + np.random.normal(0, 0.35, 1500)
y = np.digitize(score, [1.1, 1.7])                  # 3 crop categories: 0, 1, 2

def build(regularize):
    reg = regularizers.l2(1e-3) if regularize else None
    layers_list = [layers.Input(shape=(4,)),
                   layers.Dense(128, activation="relu", kernel_regularizer=reg)]
    if regularize: layers_list.append(layers.Dropout(0.3))
    layers_list.append(layers.Dense(64, activation="relu", kernel_regularizer=reg))
    if regularize: layers_list.append(layers.Dropout(0.3))
    layers_list.append(layers.Dense(3, activation="softmax"))
    m = Sequential(layers_list)
    m.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"])
    return m

for name, reg in [("Without regularization", False), ("With L2 + Dropout", True)]:
    model = build(reg)
    cb = [callbacks.EarlyStopping(patience=10, restore_best_weights=True)] if reg else []
    h = model.fit(X, y, validation_split=0.3, epochs=150, batch_size=32, verbose=0, callbacks=cb)
    print(f"{name:24s} train acc = {h.history['accuracy'][-1]:.3f} | "
          f"val acc = {h.history['val_accuracy'][-1]:.3f}")
```

Output :

```
Without regularization   train acc = 0.756 | val acc = 0.747
With L2 + Dropout        train acc = 0.743 | val acc = 0.773
```

TASK 9 – Convolution Operation and Feature Extraction | 3 Marks

> An agricultural centre uses leaf images to identify visual signs of disease, including spots, edges and texture.
>
> a) Explain why convolutional layers are suitable for image data.
>
> b) Demonstrate one convolution operation using a small image matrix and filter, and interpret the result.
>
> c) Explain the roles of filters, stride and padding.
>
> Evidence Expected: Convolution-based feature extraction, numerical example and interpretation of filter, stride and padding.

Answers

a) Suitability of convolutional layers for images

- **Local connectivity:** disease signs such as spots, edges and texture are local patterns, and each neuron looks only at a small patch (receptive field).
- **Parameter sharing:** the same filter slides over the whole image, so a spot detector learned in one place works everywhere; this greatly reduces the number of parameters compared with a fully connected layer.
- **Translation equivariance:** a lesion is detected wherever it appears on the leaf.
- **Spatial structure is preserved:** neighbouring pixels stay together instead of being flattened into a vector.
- **Hierarchical features:** stacked layers build edges → textures → spots → whole disease patterns automatically.

b) Convolution example

Consider a 5 × 5 grey-level leaf patch whose left part is bright (value 5) and right part is dark background (value 0), and a 3 × 3 vertical-edge filter. (Deep-learning libraries compute the sliding dot product without flipping the filter, which is technically cross-correlation.)

Input image (5 × 5)

5	5	5	0	0
5	5	5	0	0
5	5	5	0	0
5	5	5	0	0
5	5	5	0	0

Filter (3 × 3)

1	0	-1
1	0	-1

1	0	-1

Output at position (1,1) (top-left window, columns 1–3): each row gives 5×1 + 5×0 + 5×(−1) = 0, so the sum is 0 + 0 + 0 = **0**. At position (1,2) (columns 2–4): each row gives 5×1 + 5×0 + 0×(−1) = 5, so the sum is 5 + 5 + 5 = **15**. At position (1,3) (columns 3–5): each row gives 5×1 + 0 + 0 = 5, so the sum is **15**. Sliding over all positions (stride 1, no padding) gives a 3 × 3 feature map:

Feature map (3 × 3)

0	15	15
0	15	15
0	15	15

Interpretation: the filter responds with a value of 0 in the uniform bright region and with a large value (15) where the window covers the brightness change, so the feature map highlights a vertical edge. In a real network, learned filters detect leaf-vein edges, lesion boundaries, spots or textures in the same way.

c) Roles of filters, stride and padding

- **Filters (kernels):** small matrices of learnable weights; each filter detects one kind of pattern and produces one feature map. Using K filters produces K feature maps (output channels).
- **Stride:** the number of pixels the filter moves at each step. A larger stride skips positions, reducing the output size and computation (down-sampling) but losing fine detail.
- **Padding:** adding a border of zeros around the image. “Valid” (no padding) shrinks the output; “same” padding keeps the output size equal to the input and lets border pixels take part in the same number of computations.

$$\text{Output size} = \lfloor (n - f + 2p) / s \rfloor + 1$$

where n = input size, f = filter size, p = padding and s = stride. For the example: n = 5, f = 3, p = 0, s = 1 gives 3 × 3; with p = 1 it gives 5 × 5; with s = 2 and p = 0 it gives 2 × 2.

Code:

```
import numpy as np

image = np.array([[5, 5, 5, 0, 0]] * 5)        # bright leaf area | dark background
kernel = np.array([[1, 0, -1]] * 3)            # vertical-edge filter

def conv2d(img, ker, stride=1, pad=0):
    if pad:
        img = np.pad(img, pad)                 # zero padding
    k = ker.shape[0]
    n = (img.shape[0] - k) // stride + 1       # output size
```

```
    out = np.zeros((n, n))
    for i in range(n):
        for j in range(n):
            patch = img[i*stride:i*stride + k, j*stride:j*stride + k]
            out[i, j] = np.sum(patch * ker)
    return out

print("Feature map (stride 1, no padding):\n", conv2d(image, kernel))
print("Output shape, stride 2, no padding :", conv2d(image, kernel, stride=2).shape)
print("Output shape, stride 1, padding = 1:", conv2d(image, kernel, pad=1).shape)
```

Output :

```
========== RESTART: C:/All Folders/Deep Learning/lab1.py ==========
Feature map (stride 1, no padding):
 [[ 0. 15. 15.]
 [ 0. 15. 15.]
 [ 0. 15. 15.]]
Output shape, stride 2, no padding : (2, 2)
Output shape, stride 1, padding = 1: (5, 5)
>
```

TASK 10 – CNN Architecture and Pooling | 4 Marks

> A CNN is proposed to classify leaf images using the architecture: Input Image → Convolution → ReLU → Pooling → Convolution → ReLU → Pooling → Fully Connected Layer → Output.
>
> a) Explain the function of each major component.
>
> b) Explain how pooling reduces feature-map dimensions.
>
> c) Discuss how filter size, stride and number of filters affect feature maps.
>
> d) Explain how multiple convolutional layers learn increasingly complex features.
>
> Evidence Expected: CNN component explanation, pooling operation, feature-map dimensions and hierarchical feature extraction.

Answers

a) Function of each component

Component	Function
Input image	Raw leaf image (height × width × 3 colour channels), resized and normalised.
Convolution	Slides learnable filters over the image to extract local features (edges, spots, texture) and produces feature maps.
ReLU	Introduces non-linearity by setting negative values to zero; speeds up training and reduces vanishing gradients.

Component	Function
Pooling	Down-samples each feature map, reducing size and computation and making features tolerant to small shifts.
Second Conv + ReLU + Pooling	Combine the first-level features into more complex patterns and reduce the size again.
Fully connected layer	Flattened feature maps are combined to make the classification decision.
Output	Softmax (or sigmoid) gives the probability of each class, e.g. healthy / diseased.

b) How pooling reduces dimensions

Pooling slides a small window (usually 2 × 2, stride 2) over each feature map independently and replaces each window by one value: the maximum (max pooling) or the average (average pooling). It has no learnable parameters. The output size is:

$$\text{Output size} = \lfloor (n - f) / s \rfloor + 1 \qquad \text{e.g. } \lfloor (4 - 2) / 2 \rfloor + 1 = 2$$

A 4 × 4 feature map becomes 2 × 2, so 75% of the values are removed while the strongest responses are kept; the number of channels is unchanged. Example:

Feature map (4 × 4)

1	3	2	1
4	6	5	2
7	2	8	3
1	0	4	9

Max pooling gives [[6, 5], [7, 9]] (for example max(1, 3, 4, 6) = 6) and average pooling gives [[3.5, 2.5], [2.5, 6.0]]. Pooling reduces computation and memory, lowers the risk of overfitting and gives small translation invariance.

Code:

```
import numpy as np

fmap = np.array([[1, 3, 2, 1],
                 [4, 6, 5, 2],
                 [7, 2, 8, 3],
                 [1, 0, 4, 9]])

def pool2d(x, size=2, stride=2, mode="max"):
    n = (x.shape[0] - size) // stride + 1
    out = np.zeros((n, n))
    for i in range(n):
        for j in range(n):
            window = x[i*stride:i*stride + size, j*stride:j*stride + size]
```

```
            out[i, j] = window.max() if mode == "max" else window.mean()
    return out

print("Max pooling (2x2, stride 2):\n", pool2d(fmap))
print("Average pooling (2x2, stride 2):\n", pool2d(fmap, mode="avg"))
```

Output:

```
================= RESTART: C:/All Folders/Deep Learning/lab1.py ================
Max pooling (2x2, stride 2):
 [[6. 5.]
 [7. 9.]]
Average pooling (2x2, stride 2):
 [[3.5 2.5]
 [2.5 6. ]]
```

c) Effect of filter size, stride and number of filters

- **Filter size:** small filters (3 × 3) capture fine details with few parameters; large filters (7 × 7) see a wider area but need more parameters and computation. A bigger filter without padding produces a smaller feature map ($\lfloor (n - f + 2p)/s \rfloor + 1$).
- **Stride:** a larger stride gives a smaller feature map and faster computation but may skip fine details such as small spots; stride 1 keeps the most detail.
- **Number of filters:** it determines the depth (number of channels) of the output. More filters learn more different patterns and give richer representations, but increase parameters ($f \times f \times C_{in} \times K$), memory and the risk of overfitting.

Example for the proposed CNN with a 64 × 64 × 3 input, 3 × 3 filters, "same" padding and 2 × 2 pooling:

Layer	Output shape	Parameters
Input	64 × 64 × 3	0
Conv1 (16 filters) + ReLU	64 × 64 × 16	3×3×3×16 + 16 = 448
Pooling 1	32 × 32 × 16	0
Conv2 (32 filters) + ReLU	32 × 32 × 32	3×3×16×32 + 32 = 4,640
Pooling 2	16 × 16 × 32	0
Flatten	8,192	0
Fully connected / output (2, softmax)	2	8,192×2 + 2 = 16,386
Total		21,474

d) Hierarchical feature learning

Each convolutional layer works on the feature maps of the previous layer, so its receptive field in the original image grows with depth. The **first layer** learns simple features such as

edges, colour blobs and small gradients. The **second layer** combines these into textures, spots, vein patterns and corners. **Deeper layers** (in larger CNNs) combine them into complete lesion shapes and finally the pattern of a specific disease. This hierarchy from simple to complex features is learned automatically from data, without hand-crafted features, and the fully connected layer uses the final features for classification.

Code:

```
from tensorflow.keras import Sequential, layers

cnn = Sequential([
    layers.Input(shape=(64, 64, 3)),
    layers.Conv2D(16, (3, 3), padding="same", activation="relu"),   # Conv1 + ReLU
    layers.MaxPooling2D((2, 2)),                                    # Pool1
    layers.Conv2D(32, (3, 3), padding="same", activation="relu"),   # Conv2 + ReLU
    layers.MaxPooling2D((2, 2)),                                    # Pool2
    layers.Flatten(),
    layers.Dense(2, activation="softmax")                           # FC / output layer
])
cnn.summary()
```

Output of cnn.summary() :

```
================== RESTART: C:/All Folders/Deep Learning/lab1.py ================
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 64, 64, 16)	448
max_pooling2d (MaxPooling2D)	(None, 32, 32, 16)	0
conv2d_1 (Conv2D)	(None, 32, 32, 32)	4,640
max_pooling2d_1 (MaxPooling2D)	(None, 16, 16, 32)	0
flatten (Flatten)	(None, 8192)	0
dense (Dense)	(None, 2)	16,386

```
Total params: 21,474 (83.88 KB)
Trainable params: 21,474 (83.88 KB)
Non-trainable params: 0 (0.00 B)
```

TASK 11 – Comparative Evaluation of Feed-Forward and CNN Models | 4 Marks

> The centre considers Model A, a feed-forward network using numerical weather and soil features, and Model B, a CNN using crop-leaf images.
>
> a) Explain why Model A is appropriate for structured numerical data.

b) Explain why Model B is suitable for images.

c) Compare the models in input representation, feature extraction, parameter organization and computational requirements.

d) Recommend a model for each task and justify the choice.

Evidence Expected: Technical comparison, relationship between data type and architecture, computational comparison and justified model selection.

Answers

a) Why Model A suits structured numerical data

Rainfall, temperature, soil moisture and fertilizer quantity form a short, fixed-length feature vector. Each value has its own meaning and there is no spatial or sequential neighbourhood between the features, so weight sharing and local filters would offer no benefit. A fully connected feed-forward network lets every feature interact with every hidden neuron, so it can learn non-linear relationships and interactions (e.g. fertilizer is effective only when soil moisture is adequate). The model is small (about 81 parameters for the 4–8–4–1 design), fast to train and easy to deploy.

b) Why Model B suits images

A leaf image is a large grid of pixels with strong local correlation, and disease signs (spots, edges, texture, colour change) are local patterns that may appear anywhere on the leaf. Convolution with shared filters detects these patterns using few parameters, keeps the 2-D structure, gives translation invariance through pooling, and builds a hierarchy of features from edges to lesions. A CNN therefore learns the features directly from raw pixels.

c) Comparison

Aspect	Model A: Feed-forward network	Model B: CNN
Input representation	Vector of 4 numerical features (1-D, tabular)	Image tensor, e.g. 224 × 224 × 3 (2-D spatial grid with channels)
Feature extraction	Features are given; hidden layers only learn combinations and interactions of them	Features are learned automatically from pixels (edges → textures → lesions)
Parameter organisation	Dense weight matrices; every input connected to every neuron, no sharing	Small shared filters (kernels) reused across positions; local connectivity
Computational requirement	Very low; trains on a CPU in seconds or minutes	High; needs GPU, large memory and long training because of convolutions over many pixels and channels
Data requirement	Moderate (thousands of records)	Large (tens of thousands of images) or transfer learning
Output	Continuous yield (regression)	Class probability (healthy / diseased)

d) Recommendation

- **Crop-yield prediction → Model A (feed-forward network).** The inputs are structured numeric measurements, so a small, regularised dense network is accurate, cheap, fast and easy to interpret; a CNN would add cost without benefit.
- **Leaf-disease classification → Model B (CNN)**, preferably with transfer learning. Image data need spatial feature extraction; a dense network on flattened pixels would need millions of parameters, overfit and ignore spatial structure.

Thus each architecture is matched to its data type, and the two models can later be combined in a single system.

TASK 12 – Design Challenge: Optimize and Redesign the Deep Learning System | 8 Marks

> Changed Requirement
>
> The agricultural centre initially developed a feed-forward network using 10,000 crop records. It must now process 500,000 crop records and 100,000 leaf images. The existing model has high training accuracy, lower validation performance and long training time. Decide whether to retain, modify or redesign the system.

A. Impact Analysis | 2 Marks

> a) Explain how the larger numerical dataset and the addition of leaf images affect computational and architectural requirements.
>
> b) Identify the limitations of applying a feed-forward architecture directly to image data.

Answers

a) Effect of larger data and added images

- **Numerical data (10,000 → 500,000 records, 50× larger):** the cost of one epoch grows about 50 times, so full-batch training is no longer practical. Mini-batch training (e.g. 256 records), an efficient input pipeline and a GPU are needed. More data also reduces overfitting and allows a larger or deeper model, but the model still needs regularisation and monitoring.
- **Image data (100,000 leaf images):** a 224 × 224 × 3 image has 150,528 values, so the images need about 15 GB as 8-bit pixels (100,000 × 150,528 bytes) and about 60 GB as 32-bit floats. They cannot be loaded into memory at once, so data must be streamed from disk in batches (e.g. tf.data), decoded, resized and augmented on the fly. Training needs GPUs, and the convolution operations are far more expensive than the dense layers used for four numeric features.
- **Architecture:** the system now handles two different data types, so a single dense network is no longer suitable. Numerical records need a feed-forward branch and images need a convolutional branch (a CNN or pre-trained backbone); the two can remain separate models or be fused in a multi-input network.

b) Limitations of a feed-forward network on images

- **Loss of spatial structure:** flattening the image into a vector destroys the neighbourhood relations between pixels that define spots, edges and texture.
- **Too many parameters:** a first dense layer of only 256 neurons on a 224 × 224 × 3 image already needs 150,528 × 256 ≈ 38.5 million weights, which leads to heavy memory use, slow training and severe overfitting.
- **No translation invariance or weight sharing:** a lesion appearing at another position is seen as a new pattern and must be learned again.
- **No hierarchical feature learning** and poor robustness to shifts, rotation, scale and lighting changes.

B. Evaluation of the Existing Design | 2 Marks

> a) Evaluate the existing model using its training and validation performance.
>
> b) Explain whether simply increasing network depth and neuron count is necessarily the best solution.

Answers

a) Evaluation of the existing model

High training performance combined with clearly lower validation performance (for example 99% training versus 78% validation accuracy, a 21-point gap) is the signature of **overfitting / high variance**: the network has memorised its 10,000 training records and does not generalise. Training accuracy alone is therefore a misleading measure of quality; the validation (and test) results show the real performance. In addition, the long training time shows that the design is computationally inefficient and would become impractical for 500,000 records. For the images it has no suitable structure at all. The existing design is therefore **not acceptable as it is**.

b) Is a deeper or wider network the best solution?

No. The problem is high variance, not high bias (the model already fits the training data very well). Adding layers and neurons increases model capacity, which normally makes overfitting worse, and also increases the number of parameters, memory use and training time. Deeper networks may bring optimisation problems such as vanishing gradients and they show diminishing returns. Bigger models help only when the model is under-fitting. Better remedies are regularisation (L2, dropout, early stopping), batch normalisation, tuned optimisers and learning-rate schedules, using the larger dataset, data augmentation and, for images, an architecture that suits the data (a CNN).

C. Adaptation or Redesign | 2 Marks

> a) Propose a feed-forward network for numerical crop data and a CNN for leaf images.
>
> b) Recommend optimization and regularization methods such as Adam or SGD, learning-rate adjustment, dropout, L2 regularization, batch normalization and suitable activation functions.

Answers

a) Proposed architectures

Numerical branch (feed-forward network): Input(4 normalised features) → Dense(128) → BatchNorm → ReLU → Dropout(0.2) → Dense(64) → BatchNorm → ReLU → Dropout(0.2) → Dense(32, ReLU) → Dense(1, linear) for yield prediction. With 500,000 records a moderately larger network than 4–8–4–1 is justified, but regularisation is kept.

Image branch (CNN): Input 224 × 224 × 3 → Rescaling and augmentation → four blocks of [Conv 3 × 3 (32 → 64 → 128 → 256 filters) → BatchNorm → ReLU → MaxPool 2 × 2] → Global Average Pooling → Dropout(0.4) → Dense(1, sigmoid) for healthy / diseased (use Dense(n, softmax) for several diseases). Because 100,000 images are available but compute is limited, the recommended option is **transfer learning with a pre-trained EfficientNet-B0 (or ResNet-18/50)**: first train only the new classifier head, then fine-tune the upper layers with a small learning rate.

b) Optimization and regularization

Aspect	Recommendation	Reason
Optimiser	Adam ($\eta = 10^{-3}$) for the FFN and CNN from scratch; SGD with momentum 0.9 ($\eta \approx 0.01$) for fine-tuning	Adam converges quickly with little tuning; SGD with momentum often generalises slightly better at the end
Batch size	256 for numerical data; 32–64 for images	Efficient mini-batch training with enough gradient noise
Learning-rate adjustment	Warm-up then cosine decay, plus ReduceLROnPlateau	Fast start, stable convergence near the minimum
Activation	ReLU (or Leaky ReLU) in hidden layers; linear output for yield; sigmoid/softmax for classification	Avoids vanishing gradients; suits the output type
Loss	MSE (or Huber) for yield; binary/categorical cross-entropy for disease	Matches regression and classification tasks
Batch normalisation	After each Dense/Conv layer	Stabilises and accelerates training, allows larger η, mild regularisation
L2 regularisation	$\lambda = 10^{-4}$ on weights	Keeps weights small and reduces overfitting
Dropout	0.2 in the FFN, 0.3–0.5 before the CNN classifier	Prevents co-adaptation of neurons
Other	He initialisation, early stopping, data augmentation, mixed-precision training, gradient clipping	Better generalisation and faster, safer training

Code:

```
import tensorflow as tf
from tensorflow.keras import Sequential, Model, layers, regularizers

# (1) Feed-forward network for numerical crop data
ffn = Sequential([
    layers.Input(shape=(4,)),
    layers.Dense(128, kernel_regularizer=regularizers.l2(1e-4)),
    layers.BatchNormalization(), layers.Activation("relu"), layers.Dropout(0.2),
    layers.Dense(64, kernel_regularizer=regularizers.l2(1e-4)),
    layers.BatchNormalization(), layers.Activation("relu"), layers.Dropout(0.2),
    layers.Dense(32, activation="relu"),
    layers.Dense(1)                          # linear output: crop yield
], name="yield_ffn")
ffn.compile(optimizer=tf.keras.optimizers.Adam(1e-3), loss="mse", metrics=["mae"])

# (2) CNN trained from scratch for leaf images (healthy vs diseased)
def conv_block(filters):
    return [layers.Conv2D(filters, 3, padding="same", use_bias=False),
            layers.BatchNormalization(), layers.Activation("relu"),
            layers.MaxPooling2D()]

cnn = Sequential([
    layers.Input(shape=(224, 224, 3)),
    layers.Rescaling(1 / 255),
    *conv_block(32), *conv_block(64), *conv_block(128), *conv_block(256),
    layers.GlobalAveragePooling2D(),
    layers.Dropout(0.4),
    layers.Dense(1, activation="sigmoid")
], name="leaf_cnn")
cnn.compile(optimizer=tf.keras.optimizers.Adam(1e-3),
            loss="binary_crossentropy", metrics=["accuracy"])

# (3) Recommended: transfer learning with EfficientNet-B0
base = tf.keras.applications.EfficientNetB0(include_top=False, weights="imagenet",
                                            input_shape=(224, 224, 3))
base.trainable = False                       # train only the new head first
inp = layers.Input(shape=(224, 224, 3))
x = base(inp, training=False)                # EfficientNet expects 0-255 pixels
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.3)(x)
out = layers.Dense(1, activation="sigmoid")(x)
leaf_tl = Model(inp, out, name="leaf_efficientnet_b0")

ffn.summary();  cnn.summary();  leaf_tl.summary()
```

Output:



D. Evidence-Based Defence | 2 Marks

> a) Design an experiment comparing the original and redesigned systems using loss, validation performance, suitable classification/regression metrics and training time.
>
> b) Justify whether the original system should be retained, modified or redesigned based on the expected evidence.

Answers

a) Experimental design

- **Data split:** train / validation / test = 70 / 15 / 15, with the same split for both systems. Split images stratified by class, and split numerical records by field or season so that no near-duplicate data leak between the sets. The test set is used only once, at the end.

- **Systems compared:** original feed-forward network (SGD, no regularisation, fed with flattened data) versus the redesigned system (regularised FFN for yield + CNN/transfer-learning model for leaves).
- **Controls:** the same data, random seeds, number of epochs and hardware; repeat 3–5 runs with different seeds and report mean ± standard deviation.
- **Metrics – regression (yield):** training and validation MSE loss, RMSE, MAE and R^2.
- **Metrics – classification (disease):** training and validation loss, accuracy, precision, recall, F1-score, ROC-AUC and confusion matrix (recall of the diseased class is critical).
- **Efficiency:** training time per epoch and in total, epochs to convergence, number of parameters and inference time per sample.
- **Diagnostics:** training versus validation loss curves and the generalisation gap (training − validation score).

Code:

```
import time, numpy as np, tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from tensorflow.keras import Sequential, layers, regularizers, callbacks

# ---- Replace this block with the real crop records (4 features -> yield) ----
rng = np.random.default_rng(0)
X = rng.random((50000, 4)).astype("float32")
y = (3 * X[:, 0] + 2 * X[:, 2] + 1.5 * X[:, 0] * X[:, 3] - X[:, 1] ** 2
     + rng.normal(0, 0.3, 50000)).astype("float32")
# ----------------------------------------------------------------------------
X_tr, X_tmp, y_tr, y_tmp = train_test_split(X, y, test_size=0.3, random_state=1)
X_val, X_te, y_val, y_te = train_test_split(X_tmp, y_tmp, test_size=0.5, random_state=1)

def original_model():                  # deep, wide, unregularised, plain SGD
    m = Sequential([layers.Input(shape=(4,)),
                    layers.Dense(256, activation="relu"),
                    layers.Dense(256, activation="relu"),
                    layers.Dense(256, activation="relu"),
                    layers.Dense(1)])
    m.compile(optimizer=tf.keras.optimizers.SGD(0.01), loss="mse")
    return m

def redesigned_model():                # smaller, regularised, Adam + LR schedule
    m = Sequential([layers.Input(shape=(4,)),
                    layers.Dense(128, kernel_regularizer=regularizers.l2(1e-4)),
                    layers.BatchNormalization(), layers.Activation("relu"), layers.Dropout(0.2),
                    layers.Dense(64, kernel_regularizer=regularizers.l2(1e-4)),
                    layers.BatchNormalization(), layers.Activation("relu"), layers.Dropout(0.2),
                    layers.Dense(32, activation="relu"),
                    layers.Dense(1)])
    m.compile(optimizer=tf.keras.optimizers.Adam(1e-3), loss="mse")
    return m
```

```
def run(model, name, batch, cbs=()):
    t0 = time.time()
    model.fit(X_tr, y_tr, validation_data=(X_val, y_val), epochs=40,
              batch_size=batch, verbose=0, callbacks=list(cbs))
    train_time = time.time() - t0
    p_tr = model.predict(X_tr, verbose=0).ravel()
    p_val = model.predict(X_val, verbose=0).ravel()
    p_te = model.predict(X_te, verbose=0).ravel()
    print(f"{name:11s} | train MSE {mean_squared_error(y_tr, p_tr):.4f} | "
          f"val MSE {mean_squared_error(y_val, p_val):.4f} | "
          f"test RMSE {mean_squared_error(y_te, p_te) ** 0.5:.4f} | "
          f"test MAE {mean_absolute_error(y_te, p_te):.4f} | "
          f"test R2 {r2_score(y_te, p_te):.4f} | time {train_time:.1f} s")

run(original_model(), "Original", batch=32)
run(redesigned_model(), "Redesigned", batch=256, cbs=[
    callbacks.ReduceLROnPlateau(factor=0.5, patience=3),
    callbacks.EarlyStopping(patience=8, restore_best_weights=True)])
```

Output :

```
Original    | train MSE 0.0897 | val MSE 0.0904 | test RMSE 0.2999 | test MAE 0.
2408 | test R2 0.9490 | time 48.9 s
Redesigned  | train MSE 0.1246 | val MSE 0.1263 | test RMSE 0.3600 | test MAE 0.
2874 | test R2 0.9265 | time 7.5 s
>
```

b) Justification: retain, modify or redesign?

The expected evidence is that the original network shows a large gap between training and validation scores and a long training time, whereas the redesigned system shows a much smaller gap, a lower validation error, better validation F1 on the leaf images and a shorter time per epoch (because of mini-batches, Adam, GPU use and early stopping). The original system also cannot process leaf images effectively. The decision is therefore:

Component	Decision	Justification
Numerical yield model	Modify (retain the feed-forward idea)	A dense network is still the right architecture for tabular data; it only needs regularisation, batch normalisation, Adam with a learning-rate schedule and mini-batch training
Leaf-image classification	Redesign (new CNN / transfer learning)	A feed-forward network is structurally unsuitable for images
Overall system	Redesign as a two-branch system	Two data types need two matched architectures, deployed separately or fused

The original system should be retained only if the experiment showed validation performance close to its training performance and no meaningful gain from the redesign; the expected overfitting, training-time and image-data evidence argues against that.

Additional Design Considerations

> Compare SGD, mini-batch SGD, momentum, Adam and RMSProp; propose a learning-rate schedule; design label-preserving image augmentation; and compare LeNet, AlexNet, ResNet and EfficientNet for the available image size and computational budget.

Answers

1. Comparison of optimisation methods

Method	Update rule	Advantages	Disadvantages
SGD (one sample)	$\theta \leftarrow \theta - \eta\, g$	Cheap per step; noise helps escape poor minima	Very noisy path, slow overall, poor hardware use
Mini-batch SGD	$\theta \leftarrow \theta - \eta\, g_{batch}$	Good balance of speed, stability and GPU efficiency; standard choice	Needs batch size and η tuning; same η for all parameters
Momentum	$v \leftarrow \beta v + g;\ \theta \leftarrow \theta - \eta v$	Smooths noise, accelerates along consistent directions, damps oscillation	Extra hyper-parameter β; may overshoot
RMSProp	$s \leftarrow \rho s + (1-\rho)g^2;\ \theta \leftarrow \theta - \eta\, g/(\sqrt{s} + \varepsilon)$	Adaptive per-parameter step size; handles different gradient scales	No momentum; sensitive to η
Adam	$m \leftarrow \beta_1 m + (1-\beta_1)g;\ v \leftarrow \beta_2 v + (1-\beta_2)g^2;\ \theta \leftarrow \theta - \eta\, \hat{m}/(\sqrt{\hat{v}} + \varepsilon)$	Combines momentum and RMSProp; fast, robust default	More memory; can generalise slightly worse than tuned SGD

Choice: mini-batch Adam ($\eta = 10^{-3}$, batch size 256 for the numerical data and 32–64 for images) for fast and reliable training, followed by SGD with momentum 0.9 for fine-tuning the CNN if the best final accuracy is needed.

2. Learning-rate schedule

A **linear warm-up followed by cosine decay** is proposed: the learning rate rises from about 10^{-5} to $\eta_{max} = 10^{-3}$ during the first 3 epochs (avoids unstable early updates, especially with batch normalisation) and then decays smoothly to $\eta_{min} = 10^{-6}$ over the remaining epochs:

$$\eta_t = \eta_{min} + \tfrac{1}{2}(\eta_{max} - \eta_{min})(1 + \cos(\pi t/T))$$

It is combined with ReduceLROnPlateau (factor 0.5, patience 3) and early stopping on the validation loss. For fine-tuning a pre-trained CNN, a smaller rate (10^{-4} decaying to 10^{-5}) is used so that pre-trained features are not destroyed.

Code:

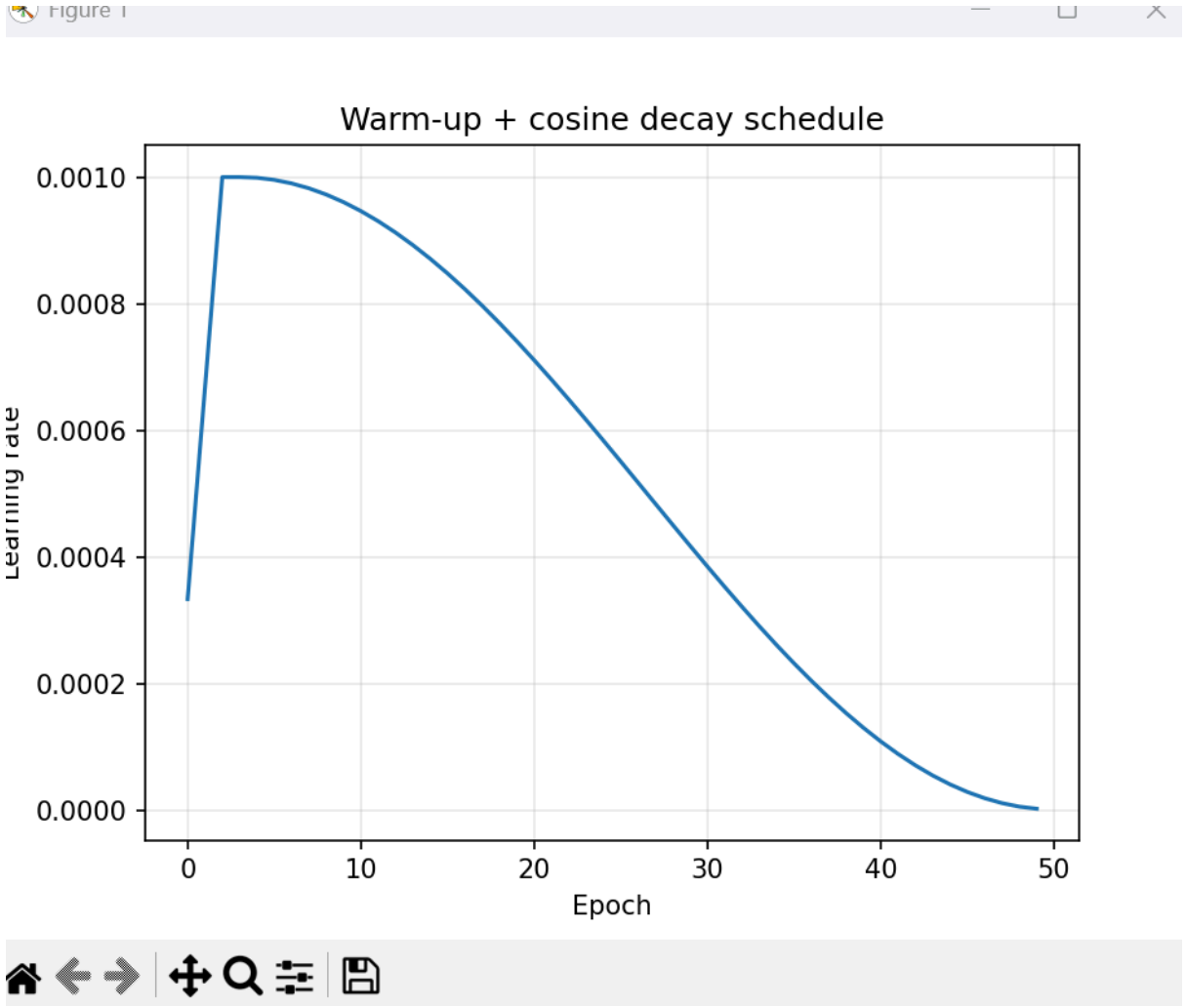
```
import numpy as np
import matplotlib.pyplot as plt

def warmup_cosine(epoch, total=50, warmup=3, lr_max=1e-3, lr_min=1e-6):
    if epoch < warmup:                          # linear warm-up
        return lr_max * (epoch + 1) / warmup
    t = (epoch - warmup) / (total - warmup)     # cosine decay
    return lr_min + 0.5 * (lr_max - lr_min) * (1 + np.cos(np.pi * t))

lrs = [warmup_cosine(e) for e in range(50)]
plt.plot(lrs);  plt.xlabel("Epoch");  plt.ylabel("Learning rate")
plt.title("Warm-up + cosine decay schedule");  plt.grid(alpha=0.3);  plt.show()

# Using it in Keras:
# lr_cb = tf.keras.callbacks.LearningRateScheduler(warmup_cosine)
# model.fit(..., callbacks=[lr_cb])
```

Output :



3. Label-preserving image augmentation

Augmentation increases the effective size and variety of the training set without changing the class label. Only transformations that keep a diseased leaf diseased (and a healthy leaf healthy) are used:

Transformation	Setting	Reason
Horizontal and vertical flip	p = 0.5	A leaf has no fixed orientation
Rotation	± 30°	Camera angle varies in the field
Zoom / random crop	0.85–1.15×, crop keeps the leaf	Different camera distance; the crop must not remove the lesion
Translation	± 10%	Leaf position varies in the frame
Brightness and contrast	± 15%	Sunlight and shadow variation
Gaussian blur / noise	Small	Camera focus and sensor noise

Avoided: strong colour/hue shifts (leaf colour such as yellowing or browning is itself a disease cue), heavy crops or cut-out that may erase the lesion, and extreme contrast changes. Augmentation is applied **only to the training set**, never to the validation or test set.

Code:

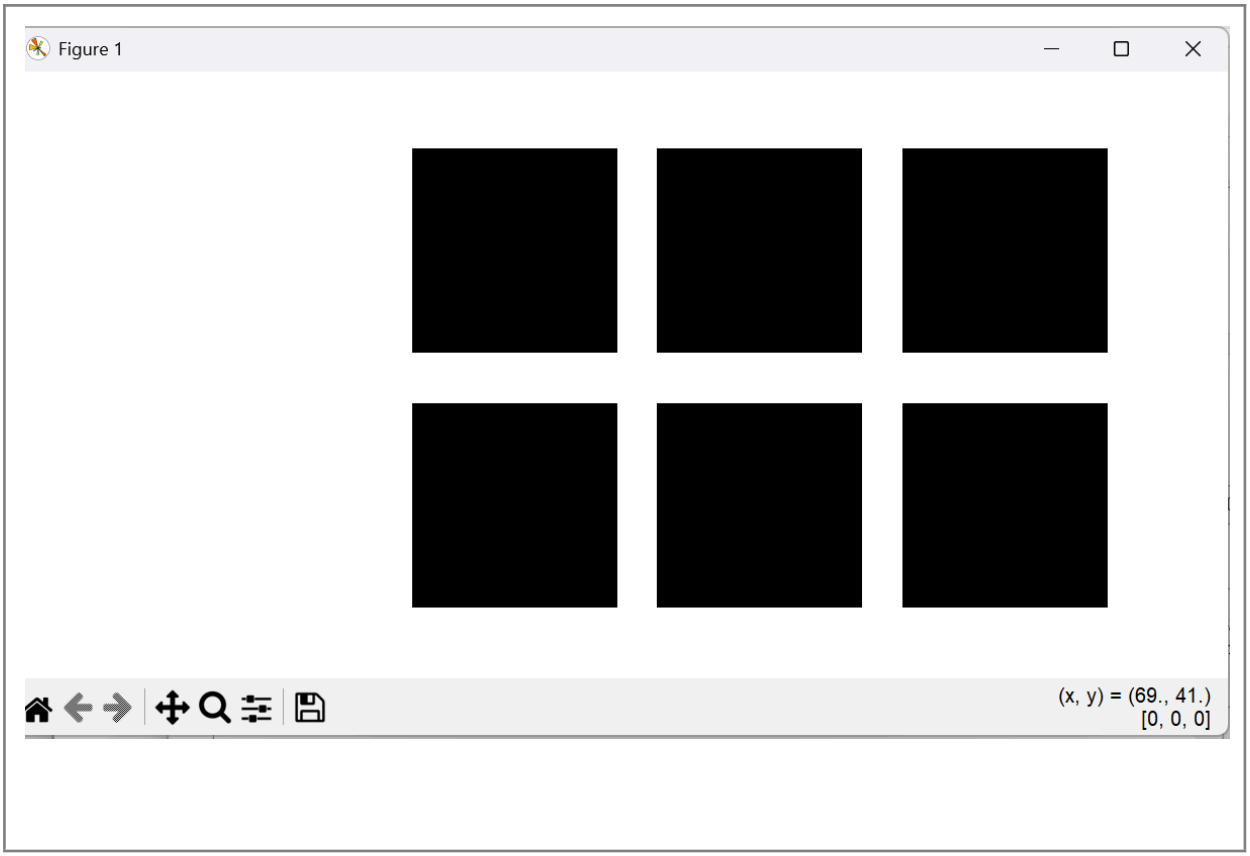
```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, Sequential

augment = Sequential([
    layers.RandomFlip("horizontal_and_vertical"),   # leaves have no fixed orientation
    layers.RandomRotation(0.08),                    # about +/- 30 degrees
    layers.RandomZoom(0.15),
    layers.RandomTranslation(0.1, 0.1),
    layers.RandomBrightness(0.15),                  # lighting variation
    layers.RandomContrast(0.15),
])

# Preview on a dummy leaf-like image (replace with a real training image)
img = np.random.rand(224, 224, 3).astype("float32")
plt.figure(figsize=(8, 4))
for i in range(8):
    plt.subplot(2, 4, i + 1)
    plt.imshow(np.clip(augment(tf.expand_dims(img, 0), training=True)[0], 0, 1))
    plt.axis("off")
plt.show()

# Use ONLY on the training set:
# train_ds = train_ds.map(lambda x, y: (augment(x, training=True), y))
```

Output – augmented samples:



4. Comparison of LeNet, AlexNet, ResNet and EfficientNet

Feature	LeNet-5	AlexNet	ResNet-18 / 50	EfficientNet-B0
Typical input	32 × 32 grey	227 × 227 colour	224 × 224 colour	224 × 224 colour
Parameters (approx.)	0.06 M	61 M	11.7 M / 25.6 M	5.3 M
Compute (approx.)	Negligible	0.7 GMACs	1.8 / 4.1 GMACs	0.39 GMACs
Key idea	Conv + pooling + small FC layers	Deep CNN with ReLU, dropout, GPU training	Residual (skip) connections and batch normalisation	Compound scaling of depth, width and resolution
ImageNet top-1 (approx.)	Not applicable	≈ 56%	≈ 70% / 76%	≈ 77%
Suitability for leaf images	Too small for detailed, colour texture images	Heavy FC layers, overfits; outdated	Very good, stable to train, widely used	Best accuracy for the compute; efficient

Recommendation: for 100,000 leaf images at about 224 × 224 (or 128–160 px if memory is limited) and a moderate GPU budget, use **EfficientNet-B0 with ImageNet pre-trained weights**; ResNet-18/50 is an equally safe and easy-to-train alternative. LeNet is only suitable for tiny 32 × 32 grey images and lacks the capacity for fine disease texture, and AlexNet needs about 61 million parameters (mostly in the dense layers) for lower accuracy.

Final conclusion

The existing feed-forward system overfits and trains slowly and cannot process images. The recommended solution is a **redesigned two-branch system**: a modified, regularised feed-forward network (Adam, batch normalisation, dropout, L2, learning-rate schedule, mini-batches) for the 500,000 numerical records, and a CNN with transfer learning (EfficientNet-B0 or ResNet, with label-preserving augmentation) for the 100,000 leaf images. The choice should be confirmed by the experiment above, using validation loss, generalisation gap, task-specific metrics and training time.